

GitHub

Guia de Organização do GitHub

VERSÃO 2.0

Projeto Final de Programação Orientada a Objetos (POO) – 2026.2

Java + Programação Orientada a Objetos + Swing + MySQL

Professor: Roger Moura Sarmiento
Instituto Federal de Educação, Ciência e Tecnologia do Ceará – IFCE

Apresentação

Bem-vindos ao Projeto Final da disciplina de Programação Orientada a Objetos (POO)! Ao longo do semestre, cada equipe representará uma Startup fictícia de desenvolvimento de software. Além de desenvolver as aplicações, a equipe deverá organizar o trabalho no GitHub de forma colaborativa, padronizada e próxima de um ambiente profissional.

Esta versão 2.0 atualiza o guia original e estabelece, principalmente, como a Organização GitHub da Startup deverá ser estruturada, quais repositórios serão obrigatórios e como os três projetos serão integrados ao final da disciplina.

Objetivo deste Guia

O objetivo principal é ajudar as equipes a organizar o repositório e o processo de desenvolvimento do Projeto Final de POO. O GitHub deverá funcionar não apenas como local de armazenamento do código, mas como registro da evolução, colaboração, documentação e integração dos projetos.

- Padronizar a Organização GitHub da Startup.
 - Separar corretamente cada projeto em seu próprio repositório.
 - Ensinar o uso adequado de commits, branches, Pull Requests e merges.
 - Facilitar a avaliação da participação dos integrantes.
 - Preparar os três projetos para uma integração final em uma única aplicação.
-



1. Estrutura Geral da Organização GitHub

A Organização deverá possuir três repositórios obrigatórios durante a fase de desenvolvimento. Cada repositório representa um projeto independente. Branches NÃO deverão ser utilizadas para separar os três projetos.

None

```
ORGANIZAÇÃO_GITHUB_DA_STARTUP/  
|  
├── 01-sistema-login  
├── 02-agenda-contatos  
└── 03-projeto-livre
```

Regra fundamental: o repositório representa um projeto/software; a branch representa uma linha de desenvolvimento ou uma funcionalidade daquele projeto.



Repositório 01 — Sistema de Login

Projeto obrigatório responsável pela autenticação inicial do usuário. Deverá possuir uma tela desenvolvida com Java/Swing e persistência local dos dados de usuário, conforme as orientações da disciplina.

- Tela de login.
- Validação das credenciais.
- Após autenticação, direcionamento para a tela/menu de seleção dos demais módulos.
- Documentação do projeto no README.md.



Repositório 02 — Agenda de Contatos

Projeto obrigatório utilizado como principal aplicação didática da disciplina. A Agenda deverá evoluir conforme as versões trabalhadas em sala e, em sua versão final, utilizar Java, Programação Orientada a Objetos, Swing e persistência em MySQL.

- Cadastro de contatos.
- Listagem e pesquisa.
- Atualização de contatos.
- Exclusão de contatos.
- Interface gráfica com Swing.
- Persistência no MySQL.
- Aplicação dos conceitos de POO trabalhados durante o semestre.

Repositório 03 — Projeto Livre

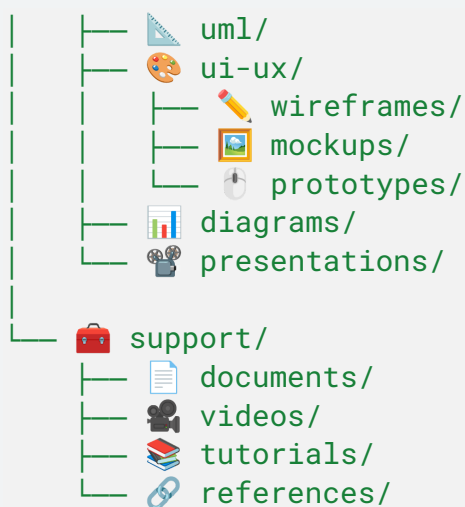
Cada equipe deverá desenvolver uma terceira aplicação de livre escolha, permitindo que os integrantes trabalhem com um tema relacionado às suas preferências e interesses, desde que o projeto seja aprovado pelo professor. Exemplos:

- Calculadora;
- Conversor;
- Chatbot;
- Consumo de API;
- Jogo;
- Urna Eletrônica;
- Java + iReport;
- Outro projeto aprovado pelo professor.

2. Estrutura Recomendada dos Repositórios

Os três repositórios deverão seguir o padrão de organização definido para os projetos da Startup.





Todos os repositórios deverão seguir, preferencialmente, a estrutura abaixo.

README.md (Raiz)

Arquivo obrigatório localizado na raiz do projeto.

O README funciona como a página principal do repositório e deverá conter informações suficientes para que qualquer pessoa consiga compreender o projeto.

Recomenda-se incluir:

-  Nome do projeto;
-  Descrição;
-  Objetivos;
-  Tecnologias utilizadas;
-  Como executar o projeto;
-  Integrantes da equipe;
-  Capturas de tela (quando existirem);
-  Organização do projeto.

2.2 Organização das Pastas e Arquivos

Pasta	Finalidade
-------	------------

src/	Código-fonte da aplicação Java. Deve concentrar classes, pacotes e demais arquivos de código do projeto.
resources/	Recursos utilizados pela aplicação, como ícones e imagens. Não é destinada a documentos de apoio.
database/	Materiais relacionados ao banco de dados: DER, Diagrama Lógico (DL) e scripts SQL.
docs/	Documentação produzida pela própria equipe: UML, protótipos, diagramas e apresentações.
support/	Materiais de apoio usados durante o desenvolvimento: documentos, vídeos, tutoriais e referências.

- ☒ README.md apresenta o projeto;
- ☒ LICENSE define a licença de uso e distribuição;
- ☒ .gitignore informa ao Git quais arquivos e diretórios locais não devem ser versionados.

Pasta **src/**

Esta pasta deverá conter **todo o código-fonte da aplicação**.

Exemplos:

- Código Java;
- Arquivos do projeto;
- Bibliotecas utilizadas;
- Recursos necessários para compilação.

Pasta **resources/**

Armazena arquivos consumidos pela aplicação durante sua execução, especialmente ícones e imagens utilizados nas interfaces Swing.

-  **resources/** — Armazena os recursos visuais utilizados diretamente pela aplicação durante sua execução.
 -  **icons/** armazena ícones utilizados nas interfaces swing;
 -  **images/** armazena o imagens utilizadas nas interfaces swing;
-

Pasta **database/**

Centraliza a modelagem e os arquivos do banco de dados.

-  **database/** — Armazena todos os arquivos relacionados à modelagem, estrutura e configuração do banco de dados.
 -  **DER/** armazena o Diagrama Entidade-Relacionamento;
 -  **DL/** armazena o Diagrama Lógico;
 -  **scripts/** armazena scripts SQL de criação, alteração e inserção de dados, quando aplicável.
-

Pasta **docs/**

Reúne a documentação produzida pela equipe sobre o próprio projeto. Inclui diagramas UML, protótipos de interface, outros diagramas e apresentações.

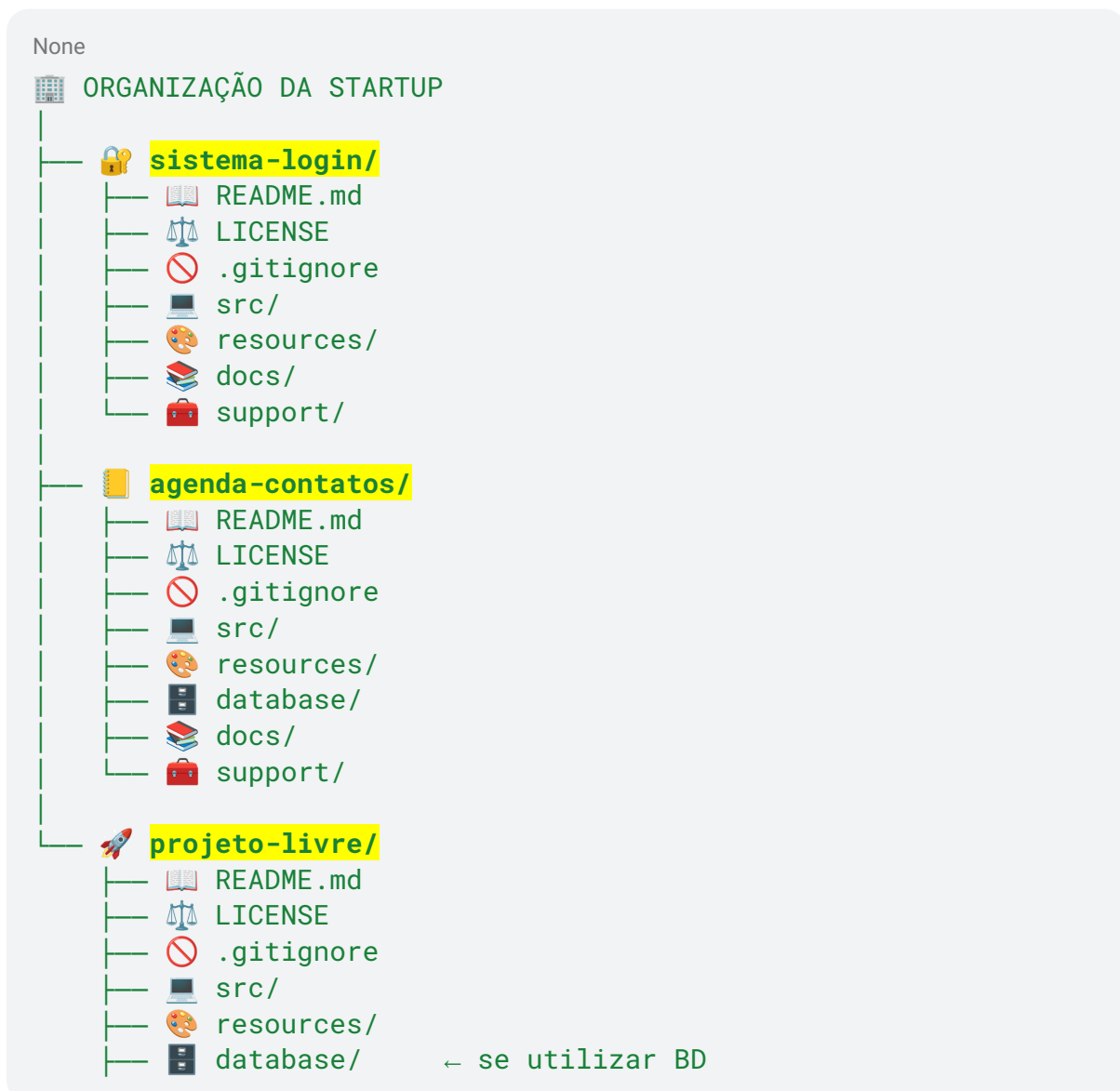
-  **docs/** — Armazena toda a documentação produzida pela equipe durante o planejamento, desenvolvimento e apresentação do projeto.
 -  **uml/** — Diagramas UML, como Classes, Casos de Uso, Sequência e Atividades.
 -  **ui-ux/** — Materiais relacionados ao planejamento e design da interface.
 -  **wireframes/** — Esboços simples da estrutura das telas.
 -  **mockups/** — Representações visuais detalhadas da aparência das telas.
 -  **prototypes/** — Protótipos que demonstram a navegação e interação entre as telas.
 -  **diagrams/** — Outros diagramas e representações visuais do projeto que não sejam UML ou de banco de dados, como fluxogramas e diagramas de arquitetura.
 -  **presentations/** — Slides e apresentações utilizados nas avaliações e demonstrações do projeto.
-

Pasta **support/**

A diferença essencial é: docs/ contém documentação do projeto; support/ contém materiais usados para ajudar a desenvolvê-lo. Para a pasta support/, seguindo o mesmo padrão sucinto:

- 📁 **support/** — Armazena materiais externos e auxiliares utilizados pela equipe como apoio durante o desenvolvimento do projeto.
 - 📄 **documents/** — Documentos, apostilas, manuais, artigos e outros arquivos utilizados como material de consulta.
 - 🎥 **videos/** — Referências para vídeos e videoaulas úteis ao desenvolvimento. Preferencialmente, armazenar os links em um README.md, evitando arquivos de vídeo grandes no repositório.
 - 📖 **tutorials/** — Tutoriais, guias e passo a passo utilizados pela equipe para aprender ou configurar tecnologias do projeto.
 - 🔗 **references/** — Links, exemplos, documentações oficiais e outras referências técnicas consultadas durante o desenvolvimento.
-

📁 Estrutura Esperada





OBS.:

Estrutura padrão: todos os repositórios deverão seguir a estrutura de organização apresentada neste guia. Pastas relacionadas a recursos não utilizados pelo projeto poderão ser omitidas. Por exemplo, um projeto que não utilize banco de dados não precisará possuir a pasta database/.

LICENSE não precisa ser diferente em cada projeto; vocês podem definir uma licença padrão para todos os projetos da Startup. Já **.gitignore** e **README.md** devem existir individualmente em cada repositório, porque cada projeto possui suas próprias instruções, descrição e arquivos ignorados.

Organização dos Commits

Procurem manter um histórico organizado durante todo o semestre.

Boas práticas:

- ✓ Commits pequenos.
- ✓ Commits frequentes.
- ✓ Mensagens claras.

Exemplos:

None

Criada tela de Login
Implementado CRUD de Contatos
Corrigido erro de conexão com MySQL
Atualizado README
Criado DER

Evitem mensagens como:

None

teste
aaaa
update
123

corrigindo

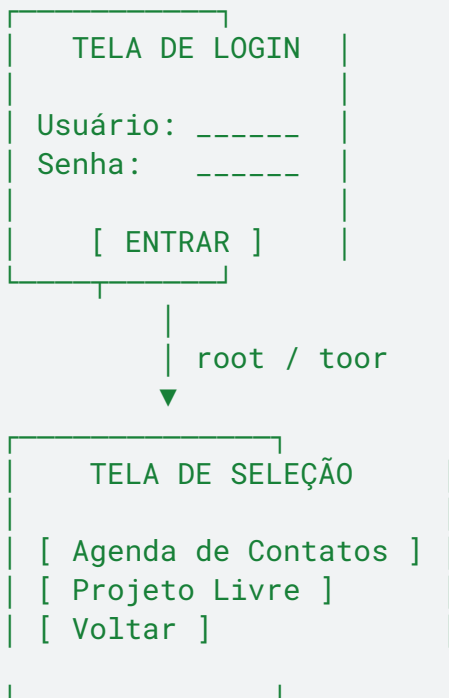


Exemplo: Organização do repositório sistema-login

1. Entendendo o que será desenvolvido

O primeiro projeto da Startup será o **Sistema de Login**. Seu funcionamento será simples:

None



Na Tela de Login, inicialmente:

None

```
Usuário: root  
Senha:  toor
```

serão verificados com uma estrutura `if`. Se estiverem corretos, a Tela de Login será fechada e a Tela de Seleção será aberta.

Na Tela de Seleção, o botão **Voltar** retorna para a Tela de Login. Os botões **Agenda de Contatos** e **Projeto Livre** serão utilizados posteriormente na integração final.

2. Criar o repositório na Organização da Startup

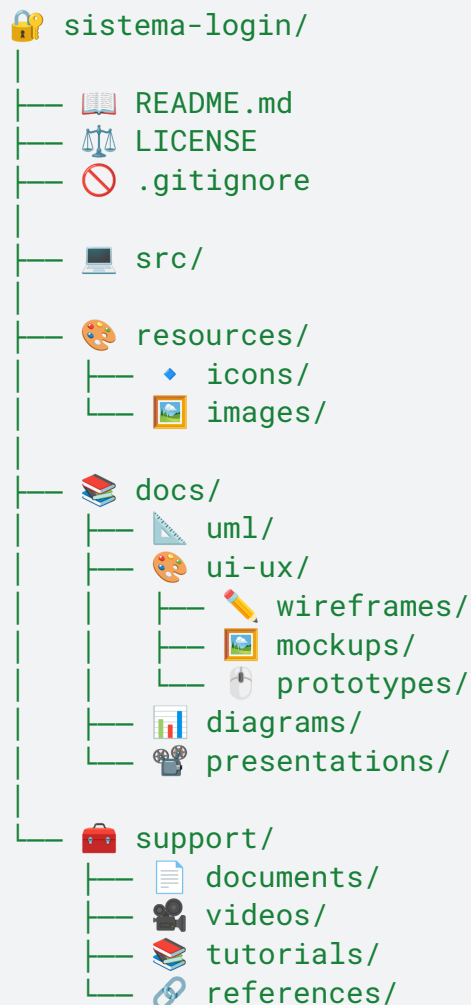
Dentro da **Organização GitHub da Startup**, criem o repositório:

None

sistema-login

A estrutura inicial será:

None



Nesse projeto **não será necessária a pasta **database/****, pois o Sistema de Login não utilizará MySQL.

3. A branch **main**

A branch:

```
None  
main
```

será a **linha principal do projeto**.

Ela deverá conter apenas funcionalidades que já foram desenvolvidas, testadas e integradas.

Portanto, a equipe deverá evitar desenvolver diretamente na **main**.

A regra será:

**Desenvolver em uma branch → testar → commit → push → Pull Request
→ merge na **main**.**

4. Criar as branches de desenvolvimento

Para esse pequeno projeto serão utilizadas duas branches:

```
None  
main  
|  
├── feature/tela-login  
└── feature/tela-selecao
```

A primeira será responsável pela Tela de Login:

```
None  
feature/tela-login
```

A segunda:

None

```
feature/tela-selecao
```

será responsável pela Tela de Seleção.

As duas devem ser criadas **a partir da main**.

5. Atualizar a main

No terminal, entre na pasta do projeto e execute:

None

```
git switch main
```

```
git pull
```

O primeiro comando garante que estamos na **main**.

O segundo verifica se nossa **main** local está atualizada com o GitHub.

6. Criar feature/tela-login

Execute:

None

```
git switch -c feature/tela-login
```

O parâmetro **-c** significa que queremos **criar uma nova branch e já mudar para ela**.

Confira:

None

```
git branch
```

O resultado deverá ser semelhante a:

None

```
* feature/tela-login
```

```
main
```

O * indica a branch em que você está trabalhando.

Agora publique essa branch no GitHub:

None

```
git push -u origin feature/tela-login
```

7. Criar **feature/tela-selecao**

Não devemos criar a segunda branch estando dentro da primeira.

Primeiro voltamos para:

None

```
git switch main
```

Atualizamos novamente:

None

```
git pull
```

Agora criamos:

None

```
git switch -c feature/tela-selecao
```

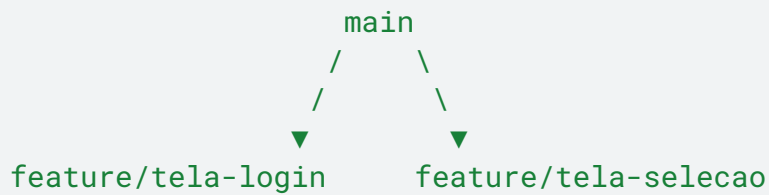
E enviamos para o GitHub:

None

```
git push -u origin feature/tela-selecao
```

Teremos:

None



As duas nasceram da mesma `main`.

8. Divisão do trabalho

Agora a equipe poderá trabalhar **em paralelo**.

Por exemplo:

Grupo	Branch	Responsabilidade
Aluno(s) A	<code>feature/tela-login</code>	Desenvolver a Tela de Login
Aluno(s) B	<code>feature/tela-selecao</code>	Desenvolver a Tela de Seleção

Isso **não** significa que a branch pertence ao aluno.

Evitem:

None

`joao`
`maria`
`branch-pedro`

A branch representa uma **funcionalidade**:

None

`feature/tela-login`
`feature/tela-selecao`

9. Trabalhando na Tela de Login

Quem ficará responsável pelo Login deverá mudar para:

None

```
git switch feature/tela-login
```

Confirme:

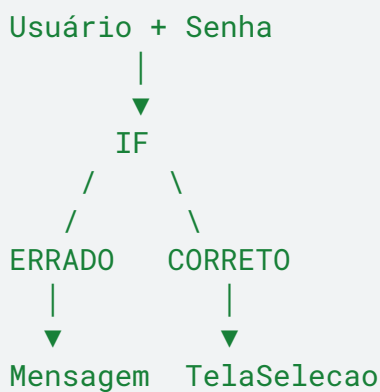
None

```
git branch
```

Agora desenvolva a Tela de Login normalmente no projeto Java/Swing.

A funcionalidade esperada é:

None



O objetivo da branch é implementar a interface e a lógica de autenticação.

10. Trabalhando na Tela de Seleção

O outro integrante poderá trabalhar simultaneamente:

None

```
git switch feature/tela-selecao
```

Nessa branch deverá ser desenvolvida:

None

```
TelaSelecao
|
├─ Agenda de Contatos
```

└─ Projeto Livre
└─ Voltar

Nesse momento, Agenda e Projeto Livre **ainda não precisam estar implementados nesse repositório.**

11. Realizar commits

Durante o desenvolvimento, não deixem tudo para um único commit.

Por exemplo:

```
None
git status
git add .
git commit -m "Criada interface da tela de login"
```

Depois:

```
None
git add .
git commit -m "Implementada validação de usuário e senha"
```

Outro exemplo:

```
None
git add .
git commit -m "Criada interface da tela de seleção"
```

Evitem:

```
None
teste
update
aaaa
123
final
final-agora-vai
```

O histórico deve permitir compreender **o que foi desenvolvido**.

12. Enviar o trabalho para o GitHub

Quem estiver trabalhando na Tela de Login executará:

```
None
git push
```

Como anteriormente utilizamos:

```
None
git push -u origin feature/tela-login
```

a branch local já está associada à branch remota. Por isso, nos próximos envios basta:

```
None
git push
```

O mesmo vale para `feature/tela-selecao`.

13. Integrar a Tela de Login

Quando `feature/tela-login` estiver concluída e testada, não vamos simplesmente abandonar a branch.

No GitHub será aberto um **Pull Request**:

```
None
feature/tela-login
|
| Pull Request
|
▼
main
```

A equipe verifica as alterações.

Estando tudo correto:

None

Pull Request



Merge



main

Agora a **main** contém a Tela de Login.

14. Atualizar a outra branch

Aqui existe uma etapa **muito importante**.

A **main** mudou.

A branch **feature/tela-selecao** foi criada **antes dessa mudança**.

Portanto, quem está trabalhando nela deve trazer a nova **main** para sua branch.

Primeiro:

None

```
git switch main  
git pull
```

Agora:

None

```
git switch feature/tela-selecao  
git merge main
```

Isso significa:

"Traga para minha branch as alterações que agora existem na **main**."

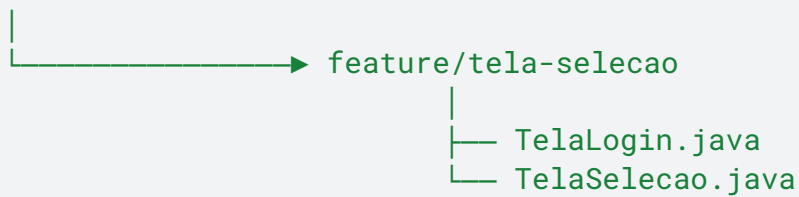
Teremos:

None

main



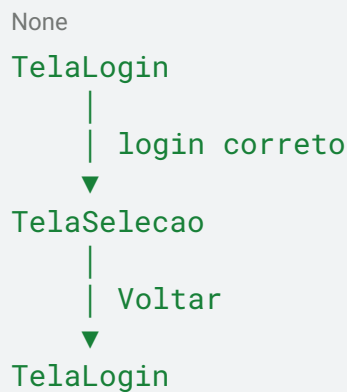
├─ TelaLogin.java



Agora as duas telas podem ser testadas juntas.

15. Fazer a navegação entre as telas

Agora podemos testar o fluxo completo:



Na Tela de Login:

```
Java
if (usuario.equals("root") && senha.equals("toor")) {
    TelaSelecao tela = new TelaSelecao();
    tela.setVisible(true);
    dispose();
}
```

Na Tela de Seleção, o botão Voltar poderá fazer:

```
Java
TelaLogin tela = new TelaLogin();
tela.setVisible(true);
dispose();
```

Agora existe efetivamente uma **integração entre as duas funcionalidades**.

16. Integrar a Tela de Seleção

Depois de testar:

None

```
git add .
git commit -m "Integrada navegação entre login e tela de seleção"
git push
```

Abra o segundo Pull Request:

None

```
feature/tela-selecao
|
| Pull Request
▼
main
```

Após a revisão:

None

Merge

Agora:

None

```
main
|
├── TelaLogin.java
└── TelaSelecao.java
```

O Sistema de Login está integrado.

17. Atualizar o projeto local

Todos os integrantes devem agora executar:

None

```
git switch main  
git pull
```

Todos passam a possuir a versão final integrada.

Podemos conferir:

None

```
git status
```

O esperado é:

None

```
On branch main  
Your branch is up to date with 'origin/main'.  
nothing to commit, working tree clean
```

18. E as branches?

Depois que o Pull Request foi aprovado e o merge realizado, as branches já cumpriram sua finalidade.

No GitHub elas podem ser excluídas.

O importante é entender que **isso não apaga os commits que foram incorporados à `main` nem o registro do Pull Request.**

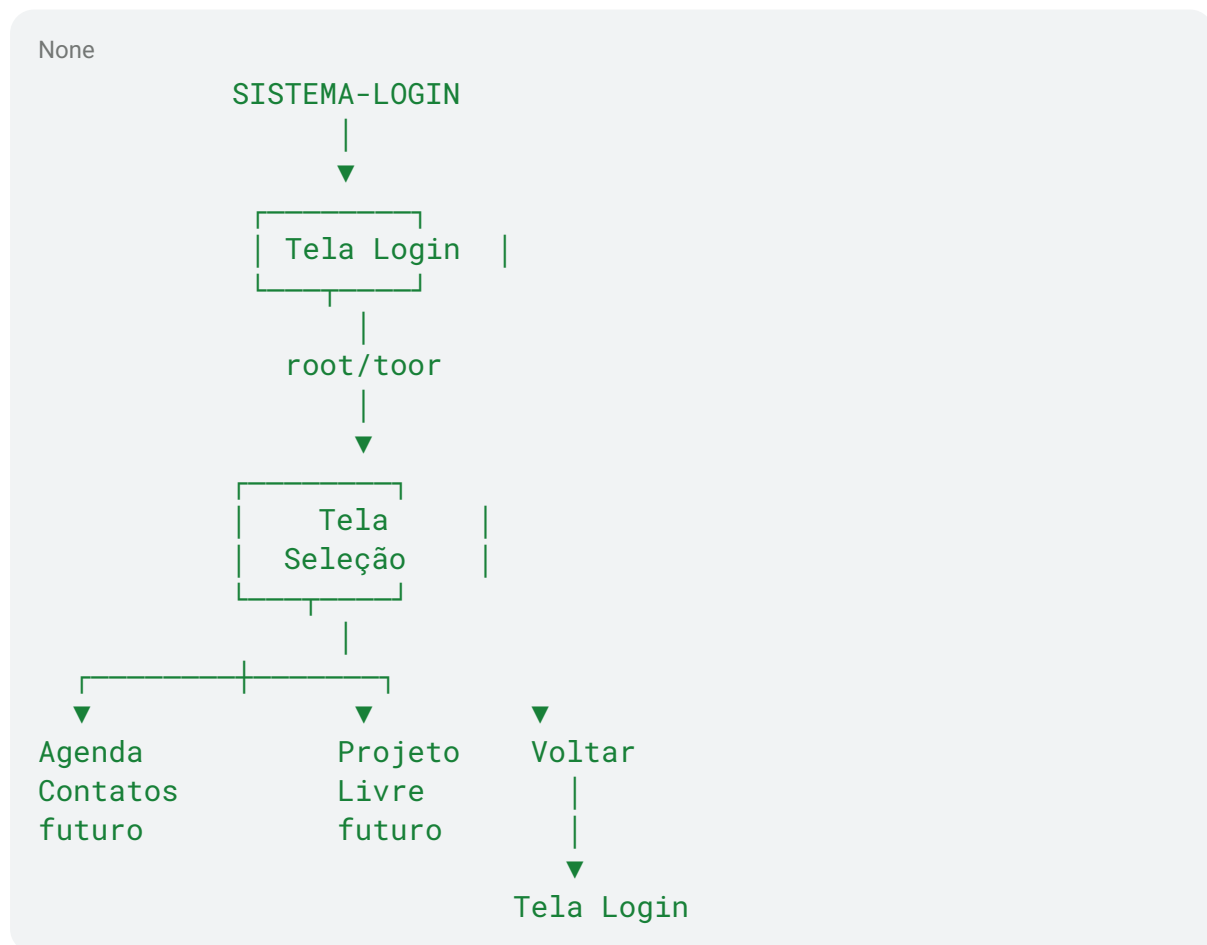
O projeto volta conceitualmente a:

None

```
🔒 sistema-login  
├──  
│   └── main  
│       ├── TelaLogin.java  
│       └── TelaSelecao.java
```

19. Resultado esperado

Ao final dessa atividade, a `main` do repositório `sistema-login` deverá executar:



A atividade, portanto, não ensina apenas a criar duas telas Swing. Ela introduz um fluxo colaborativo completo:

`main` → `feature/*` → desenvolvimento → commit → push → Pull Request → revisão → merge → `main` integrada.

Esse é o padrão que eu adotaria para o primeiro repositório antes de avançarmos para `agenda-contatos`, onde o número de funcionalidades — e consequentemente de branches — poderá ser maior.

Materiais de Apoio

Caso nunca tenha personalizado seu perfil no GitHub, os materiais abaixo podem ajudar.

Tutoriais

O que é Git e GitHub

<https://www.youtube.com/watch?v=DqTITcMq68k>

Como usar o Git e o GitHub

<https://www.youtube.com/watch?v=UBAX-13g8OM>

Como personalizar o seu perfil no Github (Readme)

<https://www.youtube.com/watch?v=TsaLQAetPLU>

Gerador de README

GitHub Profile README Generator

<https://rahuldkjain.github.io/gh-profile-readme-generator/>

★ Exemplos de Perfis

Awesome GitHub Profile README

<https://github.com/abhisheknaiidu/awesome-github-profile-readme>

Primeira Contribuição no GitHub

Caso ainda esteja aprendendo Git e GitHub, este projeto ensina como realizar sua primeira contribuição em um projeto Open Source.

First Contributions

<https://github.com/firstcontributions/first-contributions>

Checklist Final

Antes da entrega, verifiquem se a Startup possui:

- ☐ 🚩 Nome definido.
 - ☐ 🎨 Logo criado.
 - ☐ 🎯 Missão definida.
 - ☐ 📡 Visão definida.
 - ☐ ❤️ Valores definidos.
 - ☐ 👥 Integrantes cadastrados.
 - ☐ 🏠 Funções distribuídas.
 - ☐ 🐱 Organização da Startup no GitHub.
 - ☐ 📖 README da Startup criado.
 - ☐ 🏠 README de perfil criado por todos os integrantes.
 - ☐ 📁 Repositórios dos projetos criados.
 - ☐ 📁 Estrutura dos repositórios organizada.
 - ☐ 📖 README em todos os projetos.
 - ☐ 🌱 Commits realizados por todos os integrantes.
 - ☐ 📚 Documentação organizada.
-



Dicas

- ✓ Mantenham o GitHub sempre atualizado.
 - ✓ Todos os integrantes devem contribuir durante todo o semestre.
 - ✓ Façam commits frequentes.
 - ✓ Organizem bem a documentação.
 - ✓ Utilizem o GitHub como um portfólio profissional.
 - ✓ Trabalhem de forma colaborativa.
 - ✓ Não deixem para organizar o repositório apenas na semana da apresentação.
-



GitHub como Portfólio Profissional

Lembrem-se de que o GitHub é muito mais do que um local para armazenar código.

Ele poderá ser utilizado futuramente para:

- 🎓 Seleções de bolsas de pesquisa;
- 💼 Estágios;
- 🚀 Vagas de emprego;
- 📁 Portfólio profissional;
- 🤝 Colaboração em projetos Open Source.

Invistam tempo para manter seus perfis organizados e seus projetos bem documentados.

Mais do que um repositório de código, o GitHub representa a trajetória de cada desenvolvedor. Comecem a construir essa trajetória desde o primeiro semestre! 🚀